

SupaNova AI

Real-time Customer Support Helpdesk & Live
Chat System

Host Organization:	Codtech IT Solutions
Intern Name:	Naguru Suhas
Intern ID:	CITS1993
Duration:	8 Weeks
Submission Date:	June 18, 2026

Real-time Customer Support Helpdesk & Live Chat System

1. Cover Page

- **Project Title:** SupaNova AI: Real-time Customer Support Helpdesk & Live Chat System
 - **Host Organization:** Codtech IT Solutions Private Limited
 - **Intern Name:** Naguru Suhas
 - **Intern ID:** CITS1993
 - **Duration:** 8 Weeks (April - June 2026)
 - **Date of Submission:** June 18, 2026
-

2. Internship Details

- **Internship Track:** Full-Stack Web Development (Node.js & React)
 - **Intern ID:** CITS1993
 - **Name:** Naguru Suhas
 - **Duration:** 8 Weeks
 - **Reporting Authority:** Internship Division, Codtech IT Solutions Private Limited
-

3. Organization Details

- **Company Name:** Codtech IT Solutions Private Limited

- **Nature of Business:** Information Technology Consulting, Software Development, Training, and Skill Enhancement Services.
 - **Mission:** Bridging the gap between academic theory and practical corporate software development standard frameworks.
-

4. Project Abstract

SupaNova AI is a modern, responsive, full-stack customer support helpdesk and live chat platform. Built using the MERN stack (MongoDB, Express, React, Node.js), it integrates real-time communications via Socket.IO, automated ticket lifecycle routing, and a multi-tiered Role-Based Access Control (RBAC) mechanism. SupaNova AI streamlines the interface between customer support request submissions, live support agents, operational admins, and super admins. The system supports typing indicators, reading receipts, file attachments, conversation summarization, and automated ticket categorization.

5. Problem Statement

Traditional customer support systems struggle to deliver immediate, real-time feedback, leaving customers in long, opaque email ticketing queues. Furthermore, legacy helpdesks lack a clear, secure division of labor between customer-facing conversations, agent assignment lists, database operations auditing, and system configuration rules. SupaNova AI resolves these operational bottlenecks by providing immediate real-time messaging, automated routing, and role-based operational dashboards.

6. Project Objectives

- **Real-time Synchronization:** Build a bi-directional communication channel using Socket.IO to enable zero-latency messaging.
 - **Granular RBAC Enforcements:** Create secure barriers isolating Super Admins, Admins, Agents, and Customers.
 - **Context Preservation:** Provide conversation history search, file attachment support, and ratings feedback.
 - **Operational Auditing:** Log every system configuration change, role modification, or ticket status update.
 - **Production Readiness:** Optimize build artifacts for fast load times and deploy to cloud platforms (Vercel & Render).
-

7. Technology Stack

- **Frontend Core:** React, Vite (fast HMR), Tailwind CSS (sleek user interfaces), Framer Motion (micro-animations).
 - **State Management:** Zustand (lightweight client state), TanStack Query / React Query (caching and syncing).
 - **Backend Core:** Node.js, Express.js (REST API server).
 - **Real-time Engine:** Socket.IO / socket.io-client (persistent WebSocket sessions).
 - **Database & ODM:** MongoDB Atlas, Mongoose (Object Document Mapping).
 - **Security & Auth:** JSON Web Tokens (JWT), BcryptJS, Express Rate Limiter, Helmet, CORS.
 - **Testing & Tools:** Node native runner, Supertest, MongoDB Memory Server.
-

8. System Architecture

SupaNova AI implements a multi-tier client-server architecture:

1. **Presentation Layer (React Client):** Responsible for routing, auth store hydration, and rendering.
 2. **Gateway/Security Layer:** Intercepts REST requests for rate-limiting, CORS validation, and XSS sanitization.
 3. **Application Layer (Express / Socket.IO Server):** Runs the business logic and triggers real-time events.
 4. **Data Layer (MongoDB Atlas Cluster):** Persists data schemas with Mongoose ODM enforcement.
-

9. Folder Structure

``text`

supanova-ai/

```
├── client/ # React frontend
|   ├── public/ # Static files
|   ├── src/
|       ├── api/ # API client config (Axios instance)
|       ├── components/ # AppShell and ui components
|       ├── hooks/ # Custom react hooks
|       ├── pages/ # Auth, Dashboards, ChatPage, Profile
|       └── store/ # Zustand store definitions
```

```
| | └─ utils/ # Formatting helpers

| └─ package.json

| └─ tailwind.config.js

└─ server/ # Express backend

| └─ src/

| | └─ config/ # Database and middleware configs

| | └─ controllers/ # API endpoints handlers

| | └─ middleware/ # Auth checks, error handling, validation

| | └─ models/ # Mongoose collection models

| | └─ routes/ # Express routing definitions

| | └─ scripts/ # Seeding and admin creation scripts

| | └─ socket/ # WebSocket handlers

| | └─ utils/ # Role helper utilities

| └─ tests/ # Automated integration tests

└─ package.json
```

`

10. Database Design

The application utilizes six primary MongoDB collections:

- **Users:** Stores account profiles, hashed passwords, roles (`customer` , `agent` , `admin` , `super_admin`), and verification states.
 - **Conversations:** Represents support tickets. Tracks category, status, priority, participants, and assigned agents.
 - **Messages:** Stores conversation chat histories. Contains sender ID, attachment links, read indicators, and reactions.
 - **Ratings:** Customer satisfaction ratings and feedback.
 - **Notifications:** System alerts, workflow status changes, or ticket assignment notifications.
 - **AuditLogs:** Security logging tracking administrator changes.
-

11. API Design

- `POST /api/auth/register` — Registers a customer or agent.
- `POST /api/auth/login` — Authenticates a user and returns a JWT token.
- `POST /api/auth/forgot-password` — Initiates password reset via security question validation.
- `GET /api/conversations` — Fetches filtered tickets based on user role.
- `POST /api/conversations` — Submits a new ticket with automatic local classification.
- `PATCH /api/conversations/:id/status` — Updates ticket status (agents/admins only).
- `POST /api/messages/:conversationId` — Sends a message, supporting multipart uploads (files).
- `GET /api/admin/analytics` — Aggregates ticket statuses, response times, and CSAT scores (admins/super_admins only).
- `PATCH /api/admin/users/:id/role` — Restructures user roles (super_admin only).

12. Socket.IO Architecture

Real-time messaging is handled by a Socket.IO connection authenticated with JWT. Communication is compartmentalized into:

- **Connection Lifecycle:** Verification of token on handshake, binding user to individual socket room.
 - **Room Scope:** Users join explicit room channels (`conversationId`) to prevent message leakage.
 - **Event Dispatch:** Typing signals (`typing:start` / `typing:stop`) and messages (`message:send` / `message:new`) are dispatched with low latency.
-

13. Authentication Flow

1. **Register:** User inputs email, name, password, role, and chooses a security question.
 2. **Login:** User credentials verified via Bcrypt hashing comparison. Success yields a JWT signed with a server secret.
 3. **Hydration:** Client stores JWT in `localStorage` (session persistence) and appends it to subsequent request headers.
 4. **Recovery:** Forgotten passwords can be reset locally by answering the chosen security question.
-

14. RBAC Architecture

Access is checked in the backend middleware layer (`authMiddleware.js`) and frontend route wrapper (`ProtectedRoute.jsx`).

- **Customer:** Access restricted to their tickets only.
 - **Agent:** Access to assigned tickets, ticket status transitions.
 - **Admin:** View-all access, conversation monitoring, read-only analytics.
 - **Super Admin:** Exclusive rights to execute agent approvals, modify user roles, reassign tickets, and inspect system audit logs.
-

15. User Roles

1. **Customer:** Registers, creates tickets, chats, uploads attachments, and rates support.
 2. **Agent:** Receives ticket notifications, updates ticket statuses, and exchanges messages.
 3. **Admin:** Monitors conversations, analyzes performance metrics, manages users.
 4. **Super Admin:** Core manager role enforcing system configurations and permissions.
-

16. Features

- **Real-time Messaging:** Fast Socket.IO messaging with read checks and reactions.
- **Auto-Routing:** Automated ticket classification (technical, billing, sales, etc.) based on keywords.

- **Role-based Dashboards:** Sleek, custom interfaces built for each user type.
 - **Security Setup:** Recovery mechanisms and JWT validation.
 - **Dark Mode & Responsive:** Custom theme configuration, responsive CSS design.
-

17. Screenshots

(Note: These refer to the images that will be placed in the `screenshots/` directory)

- **Landing Page:** `screenshots/01_landing_page.png`
 - **Customer Chat Console:** `screenshots/05_customer_chat.png`
 - **Agent Dashboard:** `screenshots/07_agent_dashboard.png`
 - **Admin Analytics Panel:** `screenshots/14_analytics_dashboard.png`
-

18. Deployment Architecture

- **Frontend:** Hosted on Vercel as static Vite assets, served via edge CDNs.
 - **Backend:** Express API and Socket.IO server running on Render PaaS.
 - **Database:** MongoDB Atlas multi-region cluster.
-

19. Testing Results

- **Method:** Integration test suite containing assertions for auth, RBAC, workflows, and Socket.IO.

- **Result: 100% Pass Rate** (6 / 6 Test Suites passed).
 - **Average Latency:** 26 seconds total run duration.
-

20. Challenges Faced

- **WebSocket CORS Configuration:** Mismatched origins between local client/server and production domains. Resolved by implementing dynamic, environment-sensitive origin check utilities.
 - **Atlas Server Sleep:** Free-tier database and backend cold-starts. Handled by building robust loading screen transitions and error boundary retries in the React application layer.
-

21. Future Enhancements

- **AI Chatbot Agent:** Incorporate OpenAI or Gemini APIs to automatically respond to tickets before agent routing.
 - **SLA Breach Warnings:** Alert admins if a ticket remains open without agent activity for more than 24 hours.
-

22. Conclusion

SupaNova AI demonstrates how full-stack development, WebSocket integrations, and structured RBAC boundaries can be combined into a robust customer support product. Under Codtech IT Solutions, the project was successfully completed, validated, and documented.